

Краевая Соревнования Group 1

Exam ML.

[1] 1) Мы переводим модель в режим обучения `model.train()`, берем мини-батч из `DataLoader`, переносим данные на нужное устройство, делаем прогноз `pred = model(x)`, считаем функцию потерь `loss = criterion(pred, y)`, затем запускаем обратное распространение и обновляем веса. Идея такая, что `forward` считывает текущие предсказания, `loss` превращает несколько ошибок в скаляр, `loss.backward()` строит градиенты по всем параметрам через `autograd`, а `optimizer.step()` делает шаг оптимизации (как SGD/Adam), т.е. меняет на 0.

В PyTorch градиенты хранятся в `param.grad`, поэтому без `optimizer.zero_grad()` можно получить только сумму градиентов за несколько шагов обучения и обучение пойдёт.

2) Dropout: в `train` семплируем маску $m \sim \text{Bern}(1-p)$ и считаем $y = \frac{mz}{1-p}$, в `eval` $y = z$. Это регуляризация $\rightarrow$ снизить переобучение.

Batch Norm: в `train` считаем μ, σ^2 по батчу, нормируем $\bar{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \rightarrow y = \gamma \bar{x} + \beta$ в `eval` используем накоп. статистику. Переключ. в PyTorch: $\rightarrow \text{model.train}() / \text{model.eval}()$ для inference без града — `torch.no_grad()`.

[2] Полное описание в Google форме.

[3] Стационарные: $E[y_{t+1}] = \mu$ и $\text{Var}(y_t) = \sigma^2$ не зависят от t .

$\text{Cov}(y_t, y_{t-k})$ зависит от лага k . Регуляризованный = равный шаг Δt между наблюдениями.

Экзогенные переменные как внешние факторы: $y_t = f(y_{t-1}, \dots) + \beta^T x_t + \epsilon_t$, где x_t это погодные условия / факторы.

[4] Ансамбль это комбинирование предсказаний: $\hat{y}(x) = \frac{1}{M} \sum_{m=1}^M f_m(x)$ — регрессия

[5] Минимизируем внутрикластерную разбросанность, как в K-means:

$$\min(N_K) = \sum_{i=1}^n \min |x_i - \mu_K|^2$$